

1. What is Docker?

Docker is a **containerization platform** that packages an application and all its dependencies into a **container**.

A Docker container runs the same way on any operating system that supports Docker.

Example

Suppose a Python application works on your laptop but not on another system because of different library versions.

Using Docker:

- Package the application
- Include all dependencies
- Share one Docker image
- It works the same everywhere

2. Features of Docker

- Lightweight containers
- Fast deployment
- Platform independent
- Easy application sharing
- Version control for images
- Efficient resource utilization
- Isolation between applications
- Easy scaling

3. Docker Architecture

Docker consists of three main components:

Docker Client (CLI)

Accepts commands from the user.

Example:

```
docker run nginx
```

Docker Daemon (dockerd)

Runs in the background.

Responsibilities:

- Creates containers
- Builds images
- Manages networks
- Manages volumes

Docker Registry

Stores Docker images.

Examples:

- Docker Hub
- Private Registry

Docker Workflow

Docker CLI

|

V

Docker Daemon

|

V

Docker Registry

|
V

Docker Container

4. Docker Images

A Docker Image is a **read-only template** used to create containers.

Example:

Ubuntu Image

Python Image

Nginx Image

MySQL Image

Types of Images

Base Image

No parent image.

Example

ubuntu

alpine

Parent Image

Used to create another image.

Example

python:3.12

node:20

Image Layers

Every instruction in a Dockerfile creates a new layer.

Benefits

- Faster builds
- Reusability
- Less storage

Container Layer

When an image runs, Docker creates a writable layer called the **container layer**.

5. Dockerfile

A Dockerfile contains instructions to build an image.

Example

```
FROM python:3.12
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN pip install -r requirements.txt
```

```
CMD ["python","app.py"]
```

Build image

```
docker build -t myapp .
```

6. Docker Commands

Check Docker Version

`docker --version`

Docker Information

`docker info`

Download Image

`docker pull ubuntu`

List Images

`docker images`

Remove Image

`docker rmi ubuntu`

Run Container

`docker run ubuntu`

Run Container in Interactive Mode

`docker run -it ubuntu bash`

Run Container in Background

`docker run -d nginx`

Run Container with Name

`docker run --name mycontainer nginx`

Publish Ports

```
docker run -d -p 8080:80 nginx
```

Access:

<http://localhost:8080>

List Running Containers

```
docker ps
```

List All Containers

```
docker ps -a
```

Stop Container

```
docker stop mycontainer
```

Start Container

```
docker start mycontainer
```

Restart Container

```
docker restart mycontainer
```

Remove Container

```
docker rm mycontainer
```

Remove Running Container

```
docker rm -f mycontainer
```

Execute Command Inside Container

`docker exec -it mycontainer bash`

View Container Logs

`docker logs mycontainer`

Inspect Container

`docker inspect mycontainer`

Copy Files

Host → Container

`docker cp file.txt mycontainer:/app`

Container → Host

`docker cp mycontainer:/app/file.txt .`

7. Docker Storage

Docker storage is used to permanently store application data.

Without storage:

Delete Container

↓

Data Lost

With storage:

Delete Container

↓

Data Safe

Docker Volumes

Create volume

```
docker volume create myvolume
```

List volumes

```
docker volume ls
```

Inspect volume

```
docker volume inspect myvolume
```

Delete volume

```
docker volume rm myvolume
```

Use volume

```
docker run -v myvolume:/data ubuntu
```

8. Docker Networking

Docker allows containers to communicate.

Types

- Bridge
- Host
- None
- Overlay

List networks

```
docker network ls
```

Inspect network

```
docker network inspect bridge
```

Create network

```
docker network create mynetwork
```

Run container in network

```
docker run --network=mynetwork nginx
```

Delete network

```
docker network rm mynetwork
```

9. Docker Compose

Docker Compose is used to manage **multiple containers** using one YAML file.

Benefits

- One command deployment
- Easy multi-container setup
- Reusable configuration
- Simplified management

Sample docker-compose.yml

```
version: "3"
```

services:

web:

image: nginx

ports:

- "8080:80"

database:

image: mysql

environment:

MYSQL_ROOT_PASSWORD: root123

Docker Compose Commands

Start services

docker compose up

Run in background

docker compose up -d

Stop services

docker compose down

View containers

docker compose ps

View logs

docker compose logs

Restart

docker compose restart

10. Docker Engine

Docker Engine is the core component of Docker.

It consists of:

Docker CLI

User interface.

Example

```
docker run nginx
```

Docker REST API

Allows applications to communicate with Docker.

Docker Daemon

Background service that:

- Builds images
- Runs containers
- Creates networks
- Manages storage

11. Container Orchestration

Container orchestration automates the deployment, scaling, networking, and management of containers.

Why is it Needed?

- Large applications
- High availability
- Auto scaling

- Load balancing
- Automatic recovery

Benefits

- Automatic deployment
- Easy scaling
- High availability
- Load balancing
- Self-healing
- Centralized management

Kubernetes

Kubernetes is the most popular container orchestration platform.

Functions:

- Deploy containers
- Scale applications
- Restart failed containers
- Load balancing
- Rolling updates
- Auto healing

Docker vs Kubernetes

Docker

Creates and runs containers

Single-host container platform

Easy to learn

Packages applications

Kubernetes

Manages many containers

Cluster management platform

More advanced

Orchestrates applications